

Design Patterns and Design Principles Report

Resource Allocation System

Najib Attar

Contents

1	Introduction	3
2	Project Design Overview	3
3	Main GoF Patterns Applied	4
3.1	Strategy Pattern	4
3.2	Factory-Style Object Creation	5
3.3	Facade Pattern	8
3.4	Adapter Pattern	9
3.5	Singleton	11
4	GRASP Pattern Usage	11
4.1	Controller	12
4.2	Creator	12
4.3	Information Expert	12
4.4	Low Coupling and High Cohesion	12
4.5	Polymorphism	13
4.6	Pure Fabrication	14
4.7	Indirection and Protected Variation	14
5	Design Principles Usage	15
5.1	Model-View Separation	15
5.2	Low Representation Gap	15

5.3	Separation of Concerns	16
5.4	Design to Interface, Not Implementation	16
5.5	Open/Closed Principle	17
5.6	Law of Demeter	17
5.7	Find What Varies and Encapsulate It	17
5.8	Favor Composition Over Inheritance	18
5.9	Low Coupling and High Cohesion	18
6	Patterns Intentionally Not Used	19
7	How the Main Patterns Work Together	19
8	Compact Summary Tables	20
8.1	Applied GoF Patterns	20
8.2	Main Design Principles	21
9	Overengineering Discussion	21
10	Limitations and Future Opportunities	22
11	Conclusion	22
A	Short Oral Explanation	23

1 Introduction

This report explains the object-oriented design of the Resource Allocation System. The project contains a C++ backend and a Flask-based web dashboard. The backend manages users, spaces, requests, validation rules, allocation strategies, exam room sharing, committee meeting availability, data loading, result exporting, and structured summaries.

The aim of the report is not to claim that every known design pattern was used. Instead, it identifies the patterns and principles that solve real problems in the system and explains why some other patterns were intentionally not introduced.

For each major pattern, the report answers four questions:

1. What does the pattern mean?
2. Where is it used in the project?
3. What problem does it solve?
4. What benefit does it provide?

The project currently applies several GRASP responsibilities, selected GoF patterns, and general object-oriented design principles. The strongest examples are Strategy, factory-style creation, Facade, Adapter, Polymorphism, Pure Fabrication, Low Coupling, and Protected Variation.

2 Project Design Overview

The project is divided into the following main areas:

- `src/models/`: domain classes such as `User`, `Request`, `Space`, `Allocation`, and `TimeSlot`, together with their subclasses.
- `src/rules/`: independent validation rules such as `CapacityRule`, `FeatureRule`, `UserRoleRule`, `RequestTypeRule`, and `ParticipantAvailabilityRule`.
- `src/strategies/`: allocation algorithms implementing `IAllocationStrategy`.
- `src/services/`: application services such as `AllocationService`, `ResourceAllocationSession`, `UserAvailabilityService`, and `MeetingTimeSuggestionService`.
- `src/data/`: data loading and exporting components such as `DataLoader`, `DataController`, `AllocationWriter`, `RequestResultWriter`, and `SummaryWriter`.
- `web/`: Flask routes, templates, form handling, and `BackendResultAdapter`.

The main architectural separation is:

**Flask View and Input Layer → Backend Result Adapter → C++
Application and Domain Layers**

The C++ backend owns the allocation and validation behavior. Flask receives input and presents backend-generated results. Therefore, the backend can still compile and run without the web dashboard.

The following sections include focused diagrams for the main structural patterns. Separate diagrams are used because a single complete class diagram would be too dense to read clearly on an A4 page.

3 Main GoF Patterns Applied

This section explains the main GoF patterns that are actually implemented in the project.

3.1 Strategy Pattern

Meaning. The Strategy pattern defines a common interface for a family of algorithms. The client uses the interface without depending directly on one specific algorithm.

Where it is used.

- `IAllocationStrategy`
- `GreedyAllocationStrategy`
- `PriorityAllocationStrategy`
- `MultiRoomExamGreedyStrategy`
- `MultiRoomExamBestFitStrategy`
- `SharedRoomExamBestFitStrategy`
- `AllocationService`
- `AllocationStrategyFactory`

As shown in Figure 1, the session obtains the configured allocation strategy through `AllocationStrategyFactory`. `AllocationService` then delegates request processing to the common `IAllocationStrategy` interface.

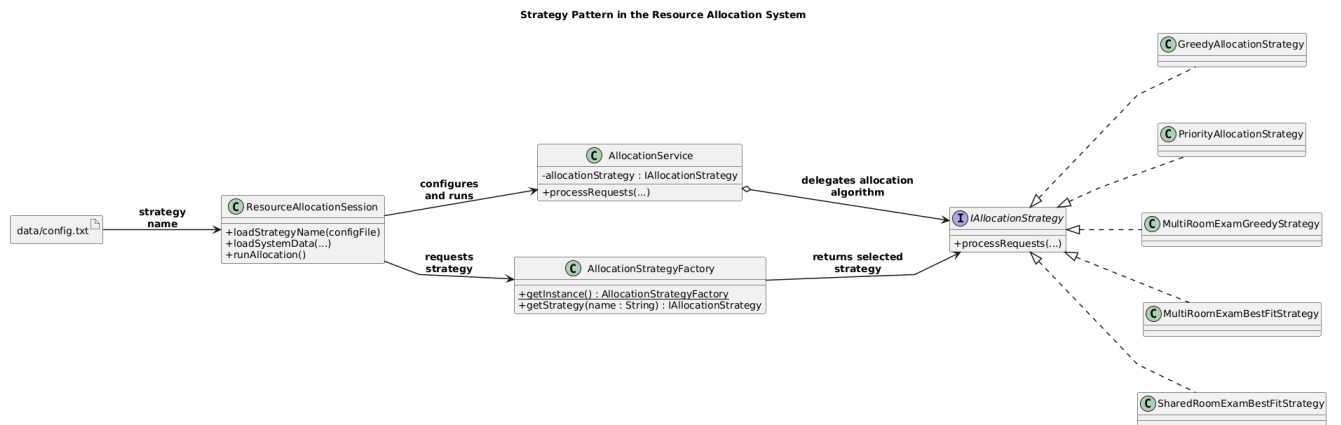


Figure 1: Strategy pattern and configuration-based algorithm selection.

Small project example.

```

const IAllocationStrategy* strategy =
    AllocationStrategyFactory::getInstance()
        .getStrategy(strategyName);

strategy->processRequests(
    requests,
    spaces,
    allocations,
    ruleEngineFacade
);

```

Why it was used. The allocation algorithm is a major variation point. The system may process requests using CSV order, priority order, multi-room best fit, or shared-room best fit.

Problem solved. Without Strategy, `AllocationService` would require many conditional statements for every allocation algorithm.

Main benefits.

- New allocation algorithms can implement the same interface.
- Existing service logic does not need to be rewritten.
- The selected strategy can be changed through configuration.
- The design supports Low Coupling and the Open/Closed Principle.

3.2 Factory-Style Object Creation

Meaning. A factory centralizes the decision about which concrete subclass should be created.

The project mainly uses static or simple factories. Therefore, it is more accurate to call this **factory-style creation** rather than strict GoF Factory Method inheritance.

UserFactory

Input: user ID, name, role name, and profile information.

Creates:

- Student
- TeachingAssistant
- Staff
- Instructor
- Administrator

RequestFactory

Input: request type, requester, requested space, participant count, metadata, and time data.

Creates:

- OneTimeRequest
- RecurringRequest
- ExamRequest
- CommitteeMeetingRequest
- InvalidRequest

Malformed or unsupported request input can be represented safely as an `InvalidRequest` instead of causing a crash.

SpaceFactory

Input: space type, ID, name, capacity, feature flags, availability, and building.

Creates:

- Classroom
- Laboratory
- MeetingRoom

Before Iteration 40, `DataLoader` directly selected and constructed the concrete space classes. After introducing `SpaceFactory`, `DataLoader` only parses and validates CSV data.

AllocationStrategyFactory

Input: strategy name from `data/config.txt`.

Returns: an implementation of `IAAllocationStrategy`.

Unknown or empty strategy names are handled using the existing safe fallback behavior.

Figure 2 shows how `DataLoader` remains responsible for reading and validating external data, while the factories decide which concrete polymorphic objects should be created.

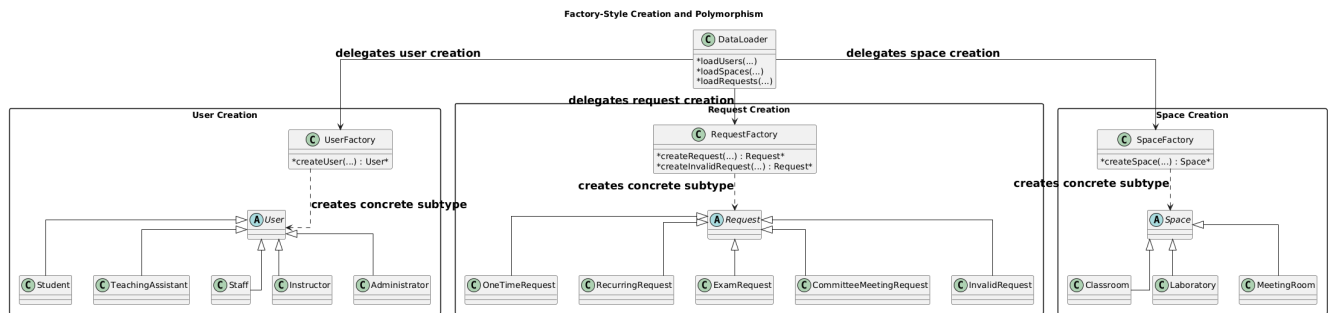


Figure 2: Factory-style creation for users, requests, and spaces.

Small project example.

```
Space* space = SpaceFactory::createSpace(  
    id,  
    name,  
    type,  
    capacity,  
    hasProjector,  
    hasWhiteboard,  
    hasComputers,  
    isAvailable,  
    building  
);
```

Problem solved. CSV parsing classes no longer need to contain all concrete object-construction decisions.

Main benefits.

- Creation logic is centralized.
- Loaders and services have fewer concrete dependencies.
- Unsupported type values are handled consistently.
- Adding a new concrete subtype has a clear extension point.

3.3 Facade Pattern

Meaning. A Facade provides a simple entry point to a more complex subsystem. Two different facades are used in the project.

ResourceAllocationSession

ResourceAllocationSession is the application-level backend facade. It coordinates:

- strategy configuration loading,
- system data loading,
- allocation execution,
- result exporting,
- structured summary exporting, and
- cleanup.

```
ResourceAllocationSession session;  
  
session.loadStrategyName("data/config.txt");  
session.loadSystemData(...);  
session.runAllocation();  
session.exportResults(...);  
session.cleanup();
```

This keeps `main.cpp` small and prepares the backend to be used by a future CLI, REST interface, desktop application, or alternative frontend.

RuleEngineFacade

RuleEngineFacade is a smaller facade focused only on rule evaluation. It hides the internal rule pipeline from allocation strategies.

The two facade levels are shown in Figure 3. **ResourceAllocationSession** simplifies the complete backend workflow, while **RuleEngineFacade** simplifies access to the rule subsystem.

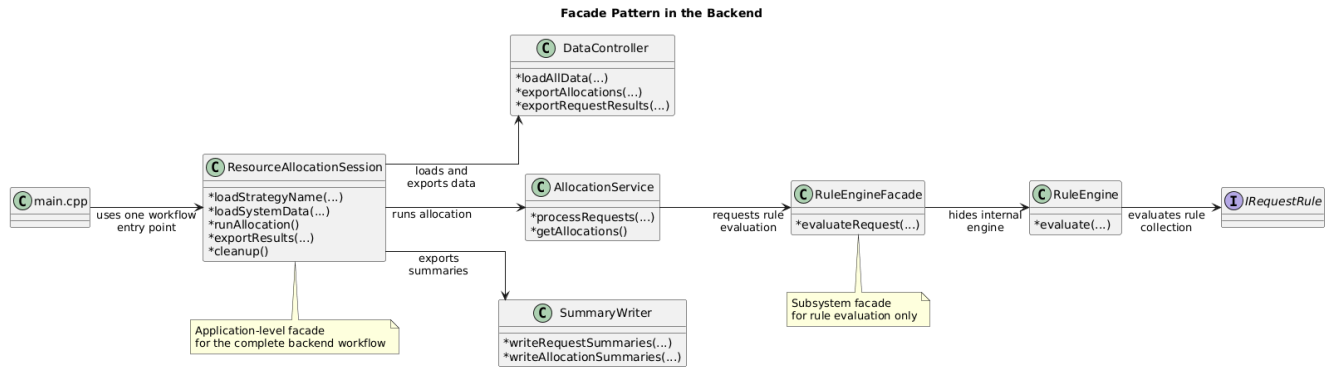


Figure 3: Application-level and subsystem-level facades.

Difference between the two facades.

Table 1: Responsibilities of the two facade classes.

Facade	Scope	Main Responsibility
ResourceAllocationSession	Full backend workflow	Load data, select strategy, run allocation, export results, and clean up.
RuleEngineFacade	Rule subsystem	Give strategies one simple method for evaluating request rules.

3.4 Adapter Pattern

Meaning. An Adapter converts an existing interface or data representation into a form that a client can use more easily.

Where it is used.

- web/backend_result_adapter.py
- BackendResultAdapter
- Flask routes in web/app.py

As shown in Figure 4, Flask routes no longer need to understand and combine every backend result file directly. They request UI-ready structures from **BackendResultAdapter**.

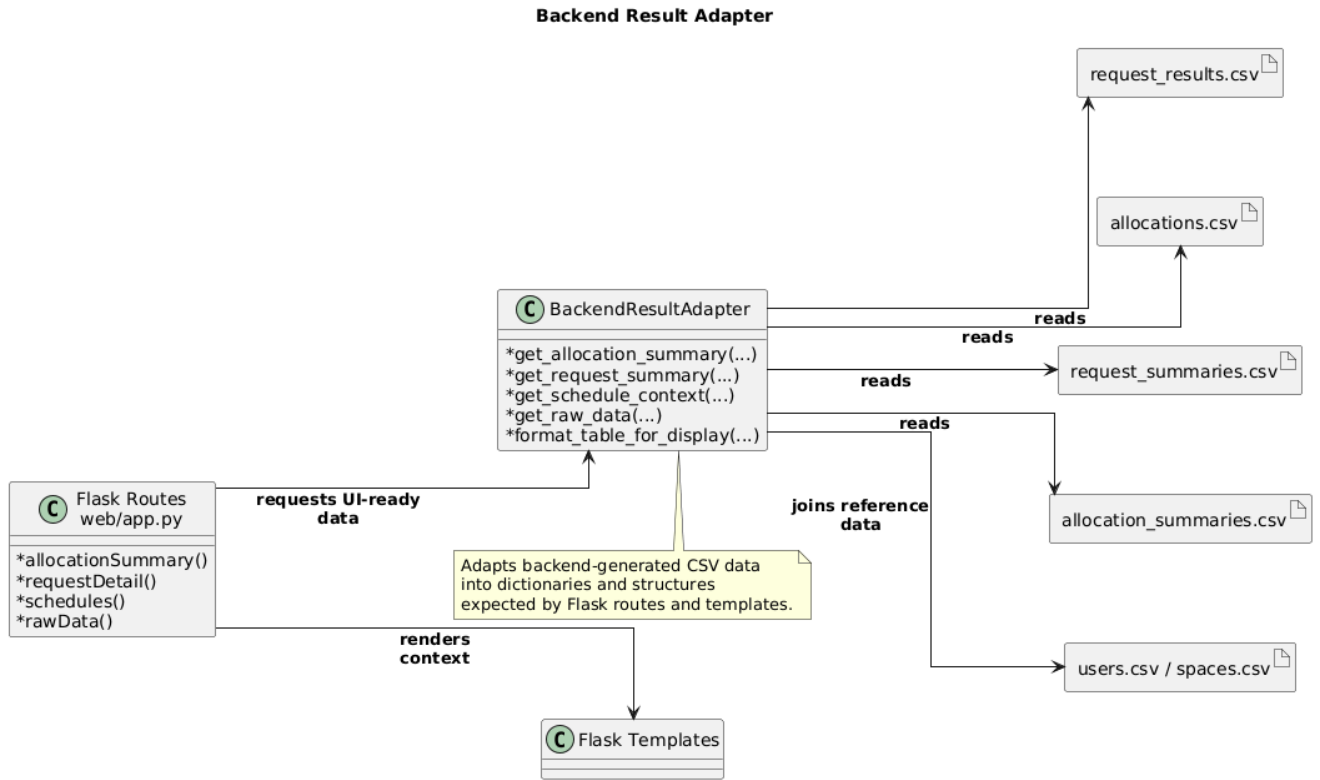


Figure 4: Adapter between Flask routes and backend-generated result files.

Small project example.

```

adapter = BackendResultAdapter(DATA_DIR)

summary = adapter.get_allocation_summary(strategy)
schedule = adapter.get_schedule_context("user", user_id)
request = adapter.get_request_summary(request_id, strategy)
  
```

Why it was used. The Flask routes previously contained repeated logic for reading CSV files, joining results, building schedules, handling missing requests, and formatting summary information.

Problem solved. The adapter hides backend output details from Flask route functions.

Main benefits.

- Flask routes remain focused on HTTP and UI flow.
- Result-reading logic is centralized.
- Changes to output interpretation affect fewer route functions.
- Schedule reconstruction and external allocation handling are consistent.
- Model–View separation is improved.

3.5 Singleton

Status: Applied narrowly.

AllocationStrategyFactory currently provides access through `getInstance()`.

```
AllocationStrategyFactory::getInstance()  
    .getStrategy(strategyName);
```

The Singleton is intentionally not used throughout the project. Excessive Singleton usage could introduce hidden global dependencies and make testing more difficult.

The current strategy factory is mostly stateless, so a static factory could also satisfy the present requirements. Therefore, the report treats Singleton as a limited implementation choice rather than a pattern that should be expanded to other classes.

4 GRASP Pattern Usage

GRASP patterns focus on assigning responsibilities to classes. The following table provides a compact overview before the detailed explanations.

Table 2: GRASP patterns used in the project.

GRASP Pat- tern	Main Project Location	Responsibility / Benefit
Controller	<code>ResourceAllocationSession</code> ; Flask routes	Coordinates system-level operations without performing all detailed work.
Creator	User, request, space, and strategy factories	Centralizes polymorphic object creation from external type values.
Information Ex- pert	<code>TimeSlot</code> , <code>User</code> , <code>Request</code> , <code>Space</code>	Assigns behavior to the class that owns the required information.
Low Coupling	Interfaces, facades, factories, adapter	Reduces the number of concrete dependencies between classes.
High Cohesion	Focused rules, writers, ser- vices, and factories	Gives each class a small and under-standable responsibility.
Polymorphism	User, request, space, rule, and strategy hierarchies	Allows different types to be handled through common abstractions.
Pure Fabrication	Controllers, services, writ- ers, factories	Keeps technical responsibilities outside domain entities.
Indirection	Facades, factories, and <code>BackendResultAdapter</code>	Adds stable boundaries between chang- ing subsystems.
Protected Varia- tion	Strategy interface, factories, adapter	Protects stable code from expected areas of change.

4.1 Controller

Primary controller: `ResourceAllocationSession`.

It receives a system-level workflow request and coordinates other components. It does not contain allocation algorithms or rule logic.

Flask routes also act as UI controllers because they receive HTTP requests, validate form input, call application helpers, and select templates.

`DataController` is better described as a technical data coordinator or Pure Fabrication rather than the main use-case controller.

4.2 Creator

The factories apply Creator by taking responsibility for constructing polymorphic objects when the concrete type depends on external input.

For example:

`CSV role string` $\longrightarrow$ `UserFactory` $\longrightarrow$ concrete `User` subclass

The same idea applies to requests, spaces, and allocation strategies.

4.3 Information Expert

Information Expert assigns behavior to the class that has the information needed to perform it.

Examples include:

- `TimeSlot::overlapsWith(...)` checks time overlap.
- `Space::hasFeature(...)` checks supported space features.
- User subclasses provide their own priority and permission behavior.
- `Request` maintains its status, rejection reason, and lifecycle history.

This avoids moving all behavior into one large manager or service class.

4.4 Low Coupling and High Cohesion

These two GRASP goals work together.

Low Coupling is supported by:

- `IAAllocationStrategy`,

- `IRequestRule`,
- factories,
- facades, and
- `BackendResultAdapter`.

High Cohesion is supported by focused classes:

- `CapacityRule` checks capacity only.
- `FeatureRule` checks features only.
- `SpaceFactory` creates spaces only.
- `SummaryWriter` writes summary outputs only.
- `MeetingTimeSuggestionService` generates time suggestions.

The result is a system in which changes usually affect a limited set of files.

4.5 Polymorphism

Polymorphism is central to the design.

Table 3: Main polymorphic hierarchies.

Base Type	Concrete Types
User	Student, TeachingAssistant, Staff, Instructor, Administrator
Request	OneTimeRequest, RecurringRequest, ExamRequest, CommitteeMeetingRequest, InvalidRequest
Space	Classroom, Laboratory, MeetingRoom
IAllocationStrategy	Greedy, priority, multi-room, best-fit, and shared-room strategies
IRequestRule	Capacity, feature, status, location, role, request type, and participant availability rules

Rule polymorphism is shown in Figure 5. `RuleEngine` evaluates rule objects through `IRequestRule`, so it does not need a separate evaluation mechanism for every concrete rule class.

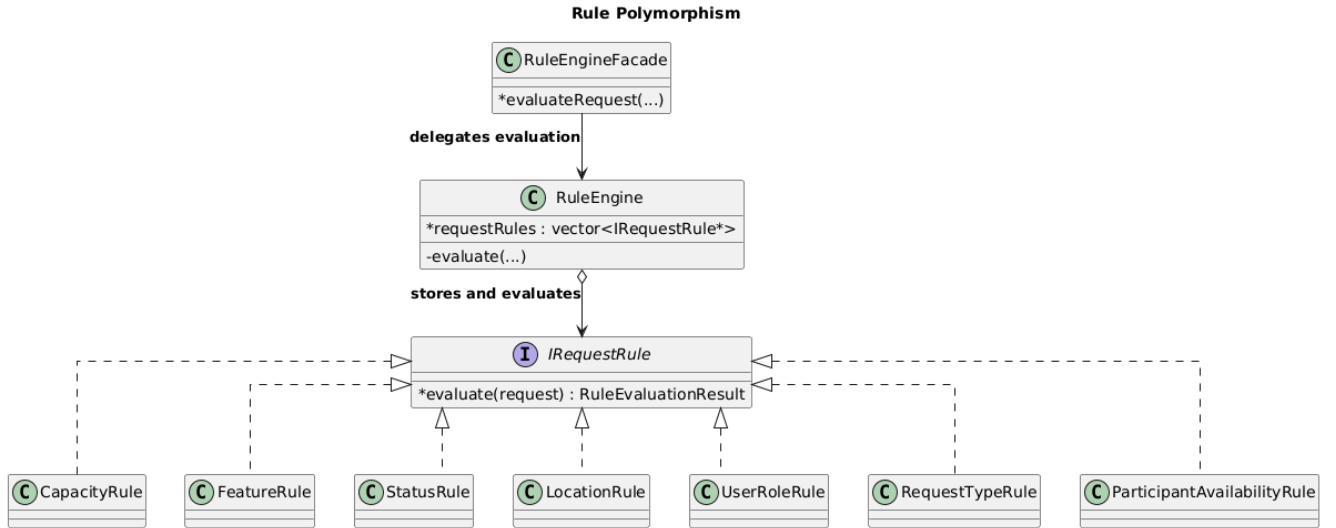


Figure 5: Polymorphic request-rule evaluation.

Polymorphism allows the rest of the system to use base pointers or interfaces while concrete objects provide different behavior.

4.6 Pure Fabrication

Pure Fabrication introduces non-domain classes when they improve the design.

Examples include:

- DataController
- DataLoader
- ResourceAllocationSession
- SummaryWriter
- MeetingTimeSuggestionService
- factory classes
- BackendResultAdapter

These are not physical university objects. They exist to keep technical responsibilities out of domain classes such as **User**, **Request**, and **Space**.

4.7 Indirection and Protected Variation

Indirection introduces an intermediate object to reduce direct dependency.

Examples:

- **RuleEngineFacade** stands between strategies and the rule pipeline.

- `BackendResultAdapter` stands between Flask routes and result files.
- factories stand between CSV/configuration strings and concrete classes.

Protected Variation identifies expected areas of change and isolates them behind stable boundaries.

Table 4: Variation points protected by the design.

Expected Variation	Protection Mechanism
Allocation algorithm	<code>IAllocationStrategy</code> and <code>AllocationStrategyFactory</code>
User subtype	<code>User</code> hierarchy and <code>UserFactory</code>
Request subtype	<code>Request</code> hierarchy and <code>RequestFactory</code>
Space subtype	<code>Space</code> hierarchy and <code>SpaceFactory</code>
Rule implementation	<code>IRequestRule</code> and <code>RuleEngineFacade</code>
Flask result interpretation	<code>BackendResultAdapter</code>

5 Design Principles Usage

5.1 Model–View Separation

Status: Mostly applied.

The C++ backend contains domain logic, validation rules, strategies, factories, services, participant availability, meeting suggestions, and result exporting.

Flask handles:

- HTTP routes,
- forms,
- redirects,
- user-facing validation messages, and
- template rendering.

`BackendResultAdapter` improves the boundary by moving result-reading and adaptation out of Flask route functions.

The separation is described as *mostly applied* rather than completely applied because Flask still understands some input CSV structures and writes CSV-based request data.

5.2 Low Representation Gap

The software classes closely represent the real university allocation domain.

Examples include:

- User
- Space
- Request
- ExamRequest
- CommitteeMeetingRequest
- Allocation
- TimeSlot

This makes the software easier to understand because its vocabulary is similar to the real problem vocabulary.

5.3 Separation of Concerns

The project separates major concerns as follows:

Table 5: Separation of concerns in the project.

Concern	Responsible Area
Domain representation	src/models/
Rule evaluation	src/rules/ and RuleEngineFacade
Allocation algorithms	src/strategies/
Workflow coordination	ResourceAllocationSession and AllocationService
Data loading and exporting	src/data/
Backend-result adaptation	BackendResultAdapter
User interface	Flask routes and templates

5.4 Design to Interface, Not Implementation

The system depends on abstractions where behavior is expected to vary.

```
const IAllocationStrategy* strategy;  
std::vector<IRequestRule*> requestRules;
```

AllocationService works with IAllocationStrategy, while the rule engine works with IRequestRule.

This allows concrete implementations to vary without changing the main client logic.

5.5 Open/Closed Principle

Status: Partially applied.

The system is open for extension through:

- new allocation strategies,
- new rules,
- new user subclasses,
- new request subclasses, and
- new space subclasses.

For example, a new allocation algorithm can implement `IAAllocationStrategy` without rewriting `AllocationService`.

The principle is only partially applied because factories still require manual registration of new type strings. This is acceptable for the current project size and keeps object creation explicit.

5.6 Law of Demeter

Status: Partially applied.

Facades and adapters reduce the number of objects that each client must know.

For example:

- `main.cpp` delegates backend workflow to `ResourceAllocationSession`.
- strategies delegate rule evaluation to `RuleEngineFacade`.
- Flask routes delegate result interpretation to `BackendResultAdapter`.

Some CSV field knowledge remains in the web input flow because requests are currently submitted through CSV-based storage.

5.7 Find What Varies and Encapsulate It

The project identifies the following major variation points:

- allocation algorithm,
- user type,
- request type,
- space type,
- rule implementation, and

- frontend interpretation of backend results.

These are encapsulated through strategies, factories, interfaces, facades, and the adapter.

5.8 Favor Composition Over Inheritance

Status: Applied with balance.

Inheritance is used where the domain naturally contains “is-a” relationships, such as:

- `Student` is a `User`,
- `ExamRequest` is a `Request`, and
- `Classroom` is a `Space`.

Composition is used to assemble application behavior.

Examples:

- `ResourceAllocationSession` composes `DataController` and `AllocationService`.
- `AllocationService` collaborates with a strategy, `RuleEngineFacade`, and meeting suggestion service.
- `RuleEngine` contains a collection of rule objects.

The project therefore favors composition for coordination while retaining inheritance where polymorphism is meaningful.

5.9 Low Coupling and High Cohesion

These principles are visible throughout the architecture.

Low Coupling means that changing one component should not require changes throughout the entire project.

High Cohesion means that each component should have a clear, focused responsibility.

Examples:

- A new strategy mainly affects the strategy package and its factory.
- A new space type mainly affects the space hierarchy and `SpaceFactory`.
- Changing schedule display logic is concentrated in `BackendResultAdapter`.
- Each rule class checks one category of condition.

6 Patterns Intentionally Not Used

Not using a pattern can be the correct design decision. The following patterns were considered but are not justified by the current requirements.

Table 6: Patterns intentionally not used.

Pattern	Reason It Is Not Used
Abstract Factory	The system does not need to create complete families of related products together. Separate factories are sufficient.
Composite	Buildings, rooms, and organization units are not currently processed through uniform tree operations.
Decorator	There is no requirement to add and remove object behavior dynamically through runtime wrappers.
Bridge	The system does not yet have two independently varying hierarchies such as several report abstractions combined with several output implementations.
Template Method	Strategy classes share some workflow ideas, but the duplication is not yet large enough to justify a rigid common algorithm skeleton.
Observer	Request status changes do not currently have independent subscribers such as notification, audit, and external integration components.

7 How the Main Patterns Work Together

The major patterns are not isolated. They cooperate in the normal backend workflow.

1. `ResourceAllocationSession` acts as the application facade and controller.
2. `DataController` and `DataLoader` read external data.
3. `UserFactory`, `RequestFactory`, and `SpaceFactory` create polymorphic domain objects.
4. `AllocationStrategyFactory` selects an `IAllocationStrategy` implementation.
5. `AllocationService` delegates the allocation algorithm to the selected strategy.
6. Strategies use `RuleEngineFacade` rather than directly managing the complete rule pipeline.
7. Writers export raw and structured result files.
8. Flask routes use `BackendResultAdapter` to transform those backend results into UI-ready data.

This collaboration can be summarized as:

External Data → Factories → Domain Objects → Strategy and Rules → Allocations →
 Writers → BackendResultAdapter → Flask Views

8 Compact Summary Tables

Instead of one oversized table, the report separates the summary into smaller tables.

8.1 Applied GoF Patterns

Table 7: Applied GoF patterns.

Pattern	Main Location	Primary Benefit
Strategy	<code>IAllocationStrategy</code> and concrete strategies	Allows allocation algorithms to vary independently.
Factory-style creation	User, request, space, and strategy factories	Centralizes polymorphic creation from external type values.
Facade	<code>ResourceAllocationSession</code> , <code>RuleEngineFacade</code>	Simplifies access to complex backend subsystems.
Adapter	<code>BackendResultAdapter</code>	Converts backend result files into UI-ready structures.
Singleton	<code>AllocationStrategyFactory</code>	Provides one shared factory access point; use remains limited.

8.2 Main Design Principles

Table 8: Main design principles applied.

Principle	Main Evidence	Result
Model–View Separation	C++ backend, Flask view layer, result adapter	Business behavior remains outside templates and routes.
Separation of Concerns	Models, rules, strategies, services, data, web	Responsibilities are divided into focused packages.
Design to Interface	<code>IAAllocationStrategy</code> , <code>IRequestRule</code>	Clients depend on abstractions where behavior varies.
Open/Closed	Extensible strategies, rules, and polymorphic hierarchies	New behavior can often be added with limited existing-code changes.
Encapsulate Variation	Strategies, factories, facades, adapter	Expected changes are isolated behind stable boundaries.
Favor Composition	Services compose collaborators	Coordination does not require excessive inheritance.
Low Coupling	Interfaces, facades, and adapter	Changes produce smaller ripple effects.
High Cohesion	Focused rule, writer, service, and factory classes	Components remain understandable and testable.

9 Overengineering Discussion

A well-designed project does not need to contain every design pattern.

Patterns were added when the project contained a real source of variation, coupling, or repeated responsibility:

- Strategy was introduced because allocation algorithms vary.
- Factories were introduced because external type strings determine concrete object types.
- Facades were introduced because backend workflow and rule evaluation needed simpler entry points.
- Adapter was introduced because Flask result parsing had become duplicated and strongly coupled to backend output files.

Patterns such as Abstract Factory, Decorator, Composite, Bridge, Template Method, and Observer were not added merely to increase the pattern count. Introducing them without a concrete requirement would increase the number of classes, interfaces, and dependency paths while providing little practical benefit.

Selective pattern usage therefore produces a clearer and more maintainable design than attempting to use every known pattern.

10 Limitations and Future Opportunities

The current design is appropriate for the project, but several future improvements remain possible:

- Flask still writes some request input directly to CSV files. A future API or command boundary could reduce this remaining coupling.
- Meeting suggestions are still represented partly through lifecycle history text. A structured suggestion export could make frontend handling cleaner.
- New factory-supported types still require explicit registration in their factory. This is acceptable for the current system size.
- If strategy classes develop substantial duplicated workflow logic, Template Method could be reconsidered.
- Observer could become useful if approvals and rejections later trigger independent email, audit, logging, or notification subscribers.
- Bridge could become relevant if report types and output formats both begin varying independently.

These are future possibilities, not current defects.

11 Conclusion

The Resource Allocation System applies design patterns and principles where they solve concrete architectural problems.

The most important applied patterns are:

- Strategy for interchangeable allocation algorithms,
- factory-style creation for polymorphic users, requests, spaces, and strategies,
- Facade for backend workflow and rule evaluation,
- Adapter for Flask-facing result access,
- Polymorphism for major domain and behavior families,
- Pure Fabrication for technical responsibilities, and
- Protected Variation for expected areas of change.

The system also demonstrates separation of concerns, low representation gap, design to interfaces, partial Open/Closed compliance, balanced use of composition and inheritance, low coupling, and high cohesion.

The project does not attempt to use every design pattern. Patterns that do not solve a current problem were intentionally excluded. This keeps the architecture extensible without making it unnecessarily complicated.

A Short Oral Explanation

The following text can be used as a short verbal explanation during a meeting or presentation.

In my project, I used design patterns only when they solved a real design problem.

The clearest example is the Strategy pattern. The system has an interface called `IAAllocationStrategy`, and different allocation algorithms implement it. The allocation service uses the interface instead of depending on one specific algorithm. This allows me to change the strategy through the configuration file and strategy factory without rewriting the service.

I also used factory-style creation for users, requests, spaces, and allocation strategies. The CSV files contain type names, and each factory converts the type name into the correct subclass. This keeps concrete construction logic out of the data loader.

For Facade, I used `ResourceAllocationSession` to provide one entry point for the full backend workflow, and `RuleEngineFacade` to provide one entry point for rule evaluation.

The Adapter pattern is used through `BackendResultAdapter`. It reads backend-generated CSV results and converts them into data that Flask routes and templates can use. Therefore, Flask does not need to duplicate all result parsing logic.

Polymorphism is also used throughout the project for users, requests, spaces, rules, and allocation strategies.

I intentionally did not use every design pattern. For example, Abstract Factory, Decorator, Composite, Bridge, Template Method, and Observer do not currently solve a necessary problem in the project. Adding them only to show more patterns would make the system harder to understand.

Therefore, the design focuses on practical extensibility, low coupling, high cohesion, and clear separation of responsibilities without unnecessary overengineering.